МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
<БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ>

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №3

Специальность АС-67

Выполнил
А. И. Петрова,
студент группы АС-67

Проверил
А. А. Крощенко,
ст. преп. кафедры ИИТ,
«___> _____2026 г.

Брест 2026

Цель: На практике сравнить работу нескольких алгоритмов классификации, таких как метод k-ближайших соседей (k-NN), деревья решений и метод опорных векторов (SVM). Научиться подбирать гиперпараметры моделей и оценивать их влияние на результат.

Вариант 7

- `Telco Customer Churn`
- Предсказать, откажется ли клиент от услуг телеком-оператора
- **Задания:**
    1. Загрузите данные, обработайте категориальные признаки и пропуски;
    2. Разделите данные на обучающую и тестовую выборки;
    3. Обучите модели k-NN, Decision Tree и SVM;
    4. Сравните модели по метрике `F1-score` для класса "отток";
    5. Сделайте вывод о применимости каждого из методов для решения бизнес-задачи по удержанию клиентов.

Выполнение:

Код программы

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

print("   Запуск задачи классификации (Telco Customer Churn)")

df = pd.read_csv("Telco-Customer-Churn.csv")

df["TotalCharges"] = pd.to_numeric(
    df["TotalCharges"].astype(str).str.strip(),
    errors="coerce"
)

X = df.drop(["customerID", "Churn"], axis=1)
y = (df["Churn"] == "Yes").astype(int)

print("Количество записей:", len(df))
print("Количество признаков:", X.shape[1])
print("Клиенты без оттока:", (y == 0).sum())
print("Клиенты с оттоком:", (y == 1).sum())

print("Пропуски TotalCharges:", X["TotalCharges"].isna().sum())

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

numeric_features = X.select_dtypes(
```

```
    include=["number"]
).columns.tolist()

categorical_features = X.select_dtypes(
    exclude=["number"]
).columns.tolist()

for column in numeric_features:
    median = X_train[column].median()

    X_train.loc[:, column] = X_train[column].fillna(median)
    X_test.loc[:, column] = X_test[column].fillna(median)

for column in categorical_features:
    mode = X_train[column].mode()[0]

    X_train.loc[:, column] = X_train[column].fillna(mode)
    X_test.loc[:, column] = X_test[column].fillna(mode)

X_train = pd.get_dummies(
    X_train,
    columns=categorical_features,
    drop_first=True,
    dtype=int
)

X_test = pd.get_dummies(
    X_test,
    columns=categorical_features,
    drop_first=True,
    dtype=int
)

X_test = X_test.reindex(
    columns=X_train.columns,
    fill_value=0
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

models = {
    "k-NN": KNeighborsClassifier(n_neighbors=5),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "SVM": SVC(kernel="rbf", C=1.0, gamma="scale")
}

results = []

for name, model in models.items():

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, zero_division=0)
    recall = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, pos_label=1, zero_division=0)

    results.append({
        "Модель": name,
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1-score": f1
    })
```

```
    print(f"\n{name}")
    print(f"Accuracy:  {accuracy:.4f}")
    print(f"Precision: {precision:.4f}")
    print(f"Recall:    {recall:.4f}")
    print(f"F1-score:  {f1:.4f}")

results = pd.DataFrame(results).sort_values(
    by="F1-score",
    ascending=False
)

print("\nСРАВНЕНИЕ МОДЕЛЕЙ")
print(results.to_string(
    index=False,
    float_format=lambda x: f"{x:.4f}"
))

print("\nЛУЧШАЯ МОДЕЛЬ")
print(results.iloc[0]["Модель"])
print(f"F1-score: {results.iloc[0]['F1-score']:.4f}")
```

Результат:

Запуск задачи классификации (Telco Customer Churn)
Количество записей: 7043
Количество признаков: 19
Клиенты без оттока: 5174
Клиенты с оттоком: 1869
Пропуски TotalCharges: 11

k-NN
Accuracy: 0.7473
Precision: 0.5253
Recall: 0.5000
F1-score: 0.5123

Decision Tree
Accuracy: 0.7410
Precision: 0.5125
Recall: 0.4920
F1-score: 0.5020

SVM
Accuracy: 0.7928
Precision: 0.6444
Recall: 0.4893
F1-score: 0.5562

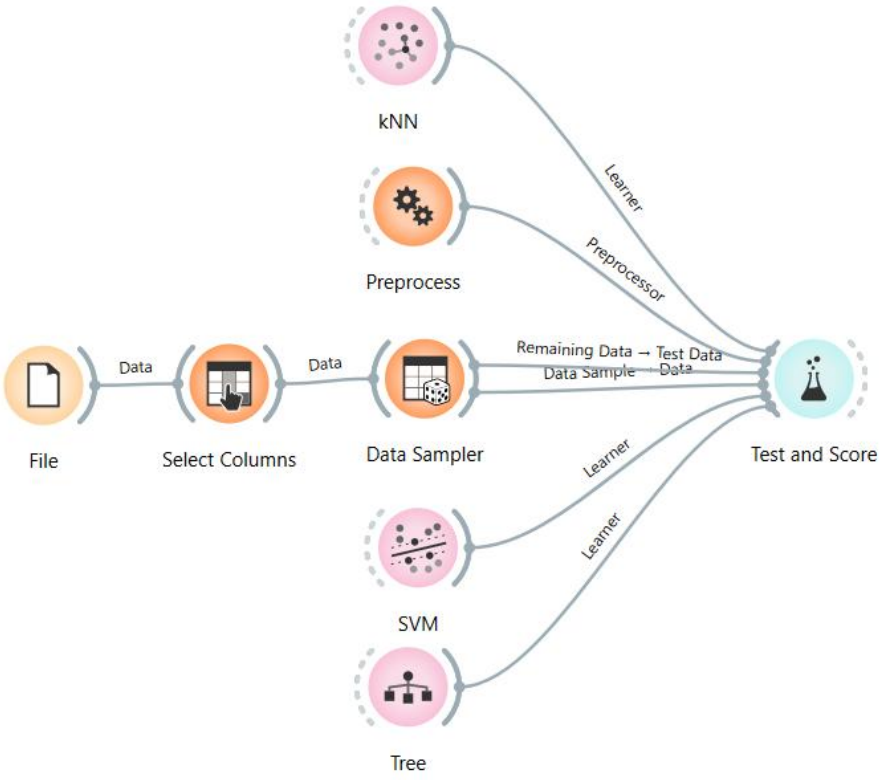
СРАВНЕНИЕ МОДЕЛЕЙ

Модель	Accuracy	Precision	Recall	F1-score
SVM	0.7928	0.6444	0.4893	0.5562
k-NN	0.7473	0.5253	0.5000	0.5123
Decision Tree	0.7410	0.5125	0.4920	0.5020

ЛУЧШАЯ МОДЕЛЬ
SVM
F1-score: 0.5562

Результаты в Orange:

Model	AUC	CA	F1	Prec	Recall	MCC
SVM	0.192	0.778	0.581	0.584	0.578	0.430
Tree	0.628	0.759	0.518	0.551	0.489	0.359
kNN	0.781	0.769	0.539	0.574	0.508	0.387



Вывод: В ходе лабораторной работы была выполнена классификация качества белого вина на два класса: обычное и хорошее. Были обучены и сравнены три алгоритма машинного обучения: k-NN, Decision Tree и SVM. Для алгоритма k-NN было подобрано оптимальное количество соседей, равное 1, при котором F1-score составил 0,7136, а Accuracy — 0,8755. Дерево решений показало F1-score 0,6356 и Accuracy 0,8327, а метод опорных векторов — F1-score 0,4444 и Accuracy 0,8265. В Orange наилучший результат также показал k-NN с F1-score 0,648. Таким образом, среди рассмотренных алгоритмов наиболее эффективным для классификации качества вина оказался метод k-ближайших соседей (k-NN).